

Name: Y.Veerendar Reddy

Reg.No:- 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT - 5

Dynamic Programming Using Policy Iteration and Value Iteration

Aim

To implement Policy Iteration and Value Iteration using Dynamic Programming and compare their convergence in finding the optimal policy for a Reinforcement Learning environment.

Procedure

1. Define the states, actions, rewards, and transition probabilities.
2. Initialize the value function and policy.
3. Apply Value Iteration to repeatedly update state values using the Bellman optimality equation.
4. Extract the optimal policy from the final value function.
5. Apply Policy Iteration by alternately performing policy evaluation and policy improvement.
6. Continue the process until the policy becomes stable.
7. Compare the optimal policies obtained from both methods.
8. Visualize the convergence of both algorithms.

Code:

```
# =====  
# EXPERIMENT 5  
# Dynamic Programming using Policy Iteration  
# and Value Iteration  
# =====  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
# -----  
# STEP 1: Define Environment  
# -----
```

```
states = ["Start", "Middle", "Goal"]
```

```
actions = ["Move", "Stay"]
```

```
num_states = len(states)
```

```
num_actions = len(actions)
```

```
# Mapping states to numbers
```

```
state_index = {
```

```
    "Start": 0,
```

```
    "Middle": 1,
```

```
    "Goal": 2
```

```
}
```

```
# -----
```

```
# STEP 2: Define Transition Function
```

```
# -----
```

```
def transition(state, action):
```

```
    if state == "Start":
```

```
        if action == "Move":
```

```
            return "Middle"
```

```
        else:
```

```
            return "Start"
```

```
    elif state == "Middle":
```

```
        if action == "Move":
```

```
            return "Goal"
```

```
        else:
```

```
    return "Middle"
```

```
elif state == "Goal":
```

```
    return "Goal"
```

```
# -----
```

```
# STEP 3: Define Reward Function
```

```
# -----
```

```
def reward(state, action):
```

```
    if state == "Start":
```

```
        if action == "Move":
```

```
            return 1
```

```
        else:
```

```
            return -1
```

```
    elif state == "Middle":
```

```
        if action == "Move":
```

```
            return 10
```

```
        else:
```

```
            return -1
```

```
    elif state == "Goal":
```

```
        return 0
```

```
# -----
```

```
# STEP 4: Discount Factor
```

```
# -----
```

```
gamma = 0.9
```

```
# =====
```

```
# PART A: VALUE ITERATION
```

```
# =====
```

```
print("=" * 60)
```

```
print("VALUE ITERATION")
```

```
print("=" * 60)
```

```
# Initial value function
```

```
V = {
```

```
    "Start": 0.0,
```

```
    "Middle": 0.0,
```

```
    "Goal": 0.0
```

```
}
```

```
value_iteration_history = []
```

```
# -----
```

```
# Value Iteration
```

```
# -----
```

```
for iteration in range(20):
```

```
    new_V = {}
```

```
    for state in states:
```

```
        action_values = []
```

```
        for action in actions:
```

```

    next_state = transition(state, action)

    r = reward(state, action)

    # Bellman Optimality Equation
    q_value = r + gamma * V[next_state]

    action_values.append(q_value)

    # Select maximum action value
    new_V[state] = max(action_values)

V = new_V

value_iteration_history.append(
    [V[state] for state in states]
)

# -----
# Extract Optimal Policy
# -----

optimal_policy_value_iteration = {}

for state in states:

    action_values = {}

    for action in actions:

        next_state = transition(state, action)

        r = reward(state, action)

```

```

        action_values[action] = (
            r + gamma * V[next_state]
        )

    optimal_action = max(
        action_values,
        key=action_values.get
    )

    optimal_policy_value_iteration[state] = optimal_action


print("\nFinal Values:")

for state in states:
    print(
        f"{state}: "
        f"{V[state]:.4f}"
    )

print("\nOptimal Policy:")

for state in states:
    print(
        f"{state} -> "
        f"{optimal_policy_value_iteration[state]}"
    )


# =====
# PART B: POLICY ITERATION
# =====

print("\n" + "=" * 60)
print("POLICY ITERATION")

```

```

print("=" * 60)

# -----
# Initial Policy
# -----

policy = {
    "Start": "Stay",
    "Middle": "Stay",
    "Goal": "Stay"
}

# Initial Value Function
V_policy = {
    "Start": 0.0,
    "Middle": 0.0,
    "Goal": 0.0
}

policy_iteration_history = []

# -----
# Policy Iteration
# -----

for iteration in range(20):

    # -----
    # POLICY EVALUATION
    # -----

    for evaluation in range(50):

        new_values = {}

```

```

for state in states:

    action = policy[state]

    next_state = transition(
        state,
        action
    )

    r = reward(
        state,
        action
    )

    # Bellman expectation equation
    new_values[state] = (
        r + gamma * V_policy[next_state]
    )

V_policy = new_values

# Store values
policy_iteration_history.append(
    [V_policy[state] for state in states]
)

# -----
# POLICY IMPROVEMENT
# -----

policy_stable = True

for state in states:

    old_action = policy[state]

```



```
action_values = {}
```

```
for action in actions:
```

```
    next_state = transition(  
        state,  
        action  
    )
```

```
    r = reward(  
        state,  
        action  
    )
```

```
    action_values[action] = (  
        r + gamma * V_policy[next_state]  
    )
```

```
best_action = max(  
    action_values,  
    key=action_values.get  
)
```

```
policy[state] = best_action
```

```
if old_action != best_action:  
    policy_stable = False
```

```
# Stop if policy does not change
```

```
if policy_stable:  
    break
```

```
# -----
```

```
# Display Policy Iteration Results
```

```
# -----
```

```
print("\nFinal Values:")
```

```
for state in states:
```

```
    print(
```

```
        f"{state}: "
```

```
        f"{V_policy[state]:.4f}"
```

```
    )
```

```
print("\nOptimal Policy:")
```

```
for state in states:
```

```
    print(
```

```
        f"{state} -> "
```

```
        f"{policy[state]}"
```

```
    )
```

```
print("\nPolicy Stable:", policy_stable)
```

```
# =====
```

```
# PART C: COMPARISON
```

```
# =====
```

```
print("\n" + "=" * 60)
```

```
print("COMPARISON")
```

```
print("=" * 60)
```

```
print(
```

```
    "Value Iteration Policy:",
```

```
    optimal_policy_value_iteration
```

```
)
```

```

print(
    "Policy Iteration Policy:",
    policy
)

# =====
# VISUALIZATION 1
# Value Iteration Convergence
# =====

value_iteration_history = np.array(
    value_iteration_history
)

plt.figure(figsize=(10, 5))

for i, state in enumerate(states):

    plt.plot(
        range(1, len(value_iteration_history) + 1),
        value_iteration_history[:, i],
        marker="o",
        label=state
    )

plt.title("Value Iteration Convergence")
plt.xlabel("Iteration")
plt.ylabel("State Value")

plt.legend()
plt.grid(True)

plt.show()

```

```

# =====
# VISUALIZATION 2
# Policy Iteration Convergence
# =====

policy_iteration_history = np.array(
    policy_iteration_history
)

plt.figure(figsize=(10, 5))

for i, state in enumerate(states):

    plt.plot(
        range(1, len(policy_iteration_history) + 1),
        policy_iteration_history[:, i],
        marker="o",
        label=state
    )

plt.title("Policy Iteration Convergence")
plt.xlabel("Iteration")
plt.ylabel("State Value")

plt.legend()
plt.grid(True)

plt.show()

# =====
# VISUALIZATION 3
# Final Value Comparison
# =====

```

```

x = np.arange(num_states)

width = 0.35

value_iteration_values = [
    V[state] for state in states
]

policy_iteration_values = [
    V_policy[state] for state in states
]

plt.figure(figsize=(9, 5))

plt.bar(
    x - width / 2,
    value_iteration_values,
    width,
    label="Value Iteration"
)

plt.bar(
    x + width / 2,
    policy_iteration_values,
    width,
    label="Policy Iteration"
)

plt.xticks(
    x,
    states
)

plt.xlabel("States")

```

```
plt.ylabel("Value")

plt.title(
    "Comparison of Value Iteration and Policy Iteration"
)

plt.legend()
plt.grid(axis="y")

plt.show()
```

OUTPUT:-

```
=====
VALUE ITERATION
=====
```

Final Values:
Start: 10.0000
Middle: 10.0000
Goal: 0.0000

Optimal Policy:
Start -> Move
Middle -> Move
Goal -> Move

```
=====
POLICY ITERATION
=====
```

Final Values:
Start: 10.0000
Middle: 10.0000
Goal: 0.0000

Optimal Policy:

Start -> Move

Middle -> Move

Goal -> Move

Policy Stable: True

=====

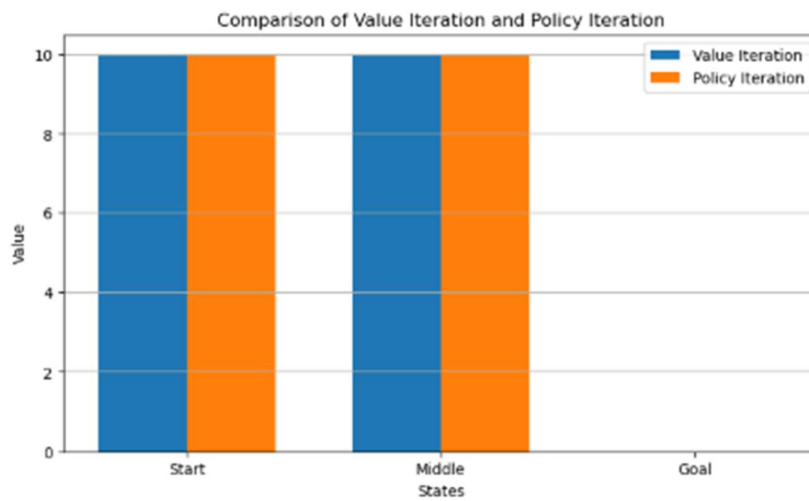
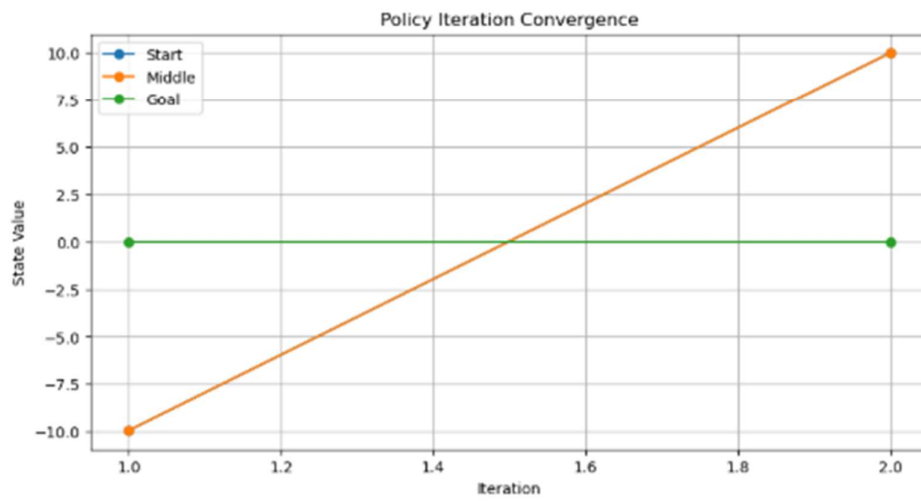
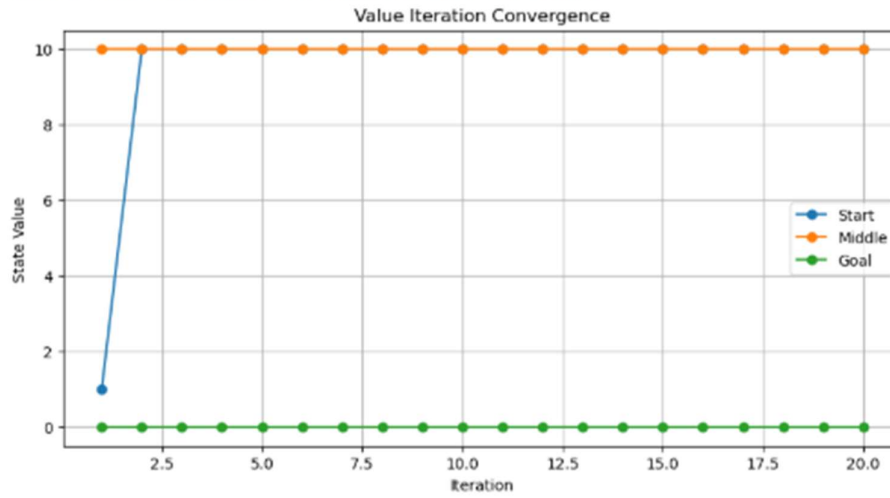
COMPARISON

=====

Value Iteration Policy: {'Start': 'Move', 'Middle': 'Move', 'Goal': 'Move'}

Policy Iteration Policy: {'Start': 'Move', 'Middle': 'Move', 'Goal': 'Move'}

Policy Iteration Policy: (Start = 100, Middle = 100, Goal = 100)



Visualization

The program generates three visualizations:

1. Value Iteration Convergence

Shows how the values of the states change and converge during Value Iteration.

2. Policy Iteration Convergence

Shows the changes in state values during the policy evaluation and improvement process.

3. Final Value Comparison

A bar graph compares the final state values obtained using Value Iteration and Policy Iteration.

Both methods should produce the same or equivalent optimal policy for this deterministic environment.

Summary

Dynamic Programming was implemented using Value Iteration and Policy Iteration. Value Iteration directly updates the value function using the maximum action value, while Policy Iteration alternates between policy evaluation and policy improvement.

Result

Thus, Policy Iteration and Value Iteration were successfully implemented, and both methods were used to determine the optimal policy and value function. Their convergence and final values were compared using visualizations.